

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 3. Atajos, Vi y comandos

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 22 de septiembre del 2026

Índice

1. Fundamentos de intercepción de eventos y atajos de teclado **1**
1.1. Captura de eventos de entrada y arquitectura del subsistema 1
1.2. Mecanismos de registro global y atajos de aplicación 1

2. Atajos de teclado en Microsoft Windows **2**
2.1. Procesamiento de mensajes y API en Win32k 2
2.2. Catálogo técnico de combinaciones esenciales 2

3. Atajos de teclado en entornos GNU/Linux **3**
3.1. Servidores gráficos: X11 y compositores Wayland 3
3.2. Catálogo de atajos en entorno GNOME sobre Wayland 5
3.3. Mecanismos y herramientas de captura de pantalla 5

4. Arquitectura y modelo modal del editor Vi **7**
4.1. Evolución histórica: de ex a POSIX.1 y Vim 7
4.2. Máquina de estados finitos y modos de ejecución 7

5. Operativa, gramática de comandos y flujo en Vi **7**
5.1. Composición sintáctica: operadores, conteos y movimientos 7
5.2. Manipulación de búferes, registros y sustitución 8

6. Demostración práctica de edición en Vim **8**
6.1. Inicialización de sesión y modo Normal 9
6.2. Captura y modificación en modo Inserción 10
6.3. Operaciones estructurales en modo Normal 10
6.4. Selección por bloques en modo Visual 10
6.5. Comandos de sustitución y persistencia en modo Ex 12

7. Estructura de directorios: rutas absolutas y relativas **12**
7.1. Jerarquía del sistema de archivos y directorio de trabajo 12
7.2. Definición formal de ruta absoluta y ruta relativa 13
7.3. Mecanismos de resolución en núcleos POSIX y Windows NT 13

8. Comandos de terminal en sistemas GNU/Linux **14**
8.1. Arquitectura del shell y ejecución de utilidades 15
8.2. Catálogo de diez comandos fundamentales 15

9. Comandos del intérprete CMD en Windows **16**
9.1. Arquitectura de cmd.exe y llamadas al sistema 18
9.2. Catálogo de diez comandos fundamentales 18

10. Análisis comparativo de interfaces y direccionamiento **20**
10.1. Comparativa entre entornos de línea de comandos 20
10.2. Diferencias operativas en la gestión de rutas 21

11. Conclusiones personales **22**

Referencias **23**

1. Fundamentos de intercepción de eventos y atajos de teclado

El teclado constituye el dispositivo primario de interacción en los sistemas operativos modernos. Desde la perspectiva del usuario, un atajo de teclado representa una pulsación simultánea o secuencial destinada a ejecutar una función expedita. Sin embargo, a nivel de arquitectura del sistema operativo, esta acción desencadena una cadena asíncrona de eventos que enlaza el hardware con el espacio de usuario (Silberschatz et al., 2006, §13.2, p. 450).

1.1. Captura de eventos de entrada y arquitectura del subsistema

La pulsación física en la matriz del teclado induce al microcontrolador integrado a sintetizar un código de escaneo binario (*scancode*), diferenciando entre el código de activación (*make code*) y el de liberación (*break code*). Esta asimetría permite al sistema detectar teclas modificadoras sostenidas (Shift, Control, Alt) (Carretero Pérez et al., 2001, §7.8, p. 399). La transferencia hacia la placa base se canaliza mediante interfaces USB HID o el controlador 8042, disparando una interrupción física (IRQ) en el controlador APIC.

Para maximizar el rendimiento, el núcleo divide la atención en dos fases: la mitad superior (*top half* o ISR mínima) almacena el scancode en una cola circular de memoria, mientras que la mitad inferior (*bottom half*, como DPC en Windows o *tasklets* en Linux) traduce el código a una clave virtual universal (Stallings, 2018, §1.4, p. 40). En GNU/Linux, el subsistema `evdev` expone estructuras `input_event` a través de `/dev/input/event*` procesadas por `libinput`; en Windows NT, el controlador `kbdclass.sys` entrega los paquetes al hilo de entrada cruda (*raw input thread*, RIT) en `win32k.sys` (Microsoft Corporation, 2023b).

1.2. Mecanismos de registro global y atajos de aplicación

Al recibir el evento normalizado, el subsistema gráfico determina si se trata de un atajo global del sistema o un acelerador local de aplicación. Los atajos globales (*global hotkeys*) actúan independientemente de la ventana enfocada. En Microsoft Windows, se registran mediante la API `RegisterHotKey`, haciendo que el hilo RIT intercepte la combinación y despache un mensaje `WM_HOTKEY` prioritario al hilo correspondiente (Microsoft Corporation, 2023b). En GNU/Linux bajo Wayland, el compositor gráfico asume el control exclusivo del teclado para prevenir el espionaje (*keylogging*) propio de la arquitectura histórica X11 (`XGrabKey`), canalizando atajos de terceros mediante el portal de escritorio XDG `org.freedesktop.portal.GlobalShortcuts` sobre D-Bus con autorización explícita del usuario (Flatpak and Freedesktop.org Contributors, 2024a).

Si una combinación no coincide con un registro global, los eventos se despachan al proceso en primer plano, donde las tablas de aceleradores (`TranslateAccelerator` en Win32) o controladores de *toolkits* (GTK, Qt) mapean las pulsaciones a comandos internos. La Tabla 1 sintetiza este flujo multidimensional en el sistema operativo.

Tabla 1: Niveles de procesamiento e intercepción de atajos en el sistema operativo.

Nivel	Componentes y flujo de señales	Resolución y alcance del atajo
Hardware / Firmware	Matriz física, microcontrolador y controlador USB HID / 8042.	Genera *scancodes* crudos y dispara interrupciones físicas (IRQ) hacia la CPU.
Núcleo del SO	Rutina ISR, mitades inferiores (DPC / *tasklets*), subsistema `evdev` / `kbdclass.sys`.	Decodifica scancodes, procesa teclas modificadoras y sintetiza códigos virtuales normalizados.
Servidor / Compositor	Subsistema Win32k (hilo RIT) o compositor Wayland / servidor X11.	Evalúa capturas globales (`RegisterHotKey`, portales XDG). Desvía atajos del entorno de escritorio.
Espacio de usuario	Bucle de mensajes del proceso enfocado y tablas de aceleradores (GTK, Qt, Win32).	Interpreta eventos dirigidos a la ventana activa (ej. `Ctrl+S`), mapeándolos a funciones internas.

2. Atajos de teclado en Microsoft Windows

En el sistema operativo Microsoft Windows, los atajos de teclado constituyen una parte integral de la experiencia de usuario y de la administración del sistema. Su funcionamiento se apoya en una arquitectura híbrida donde los controladores del núcleo, el subsistema gráfico de ventanas y el intérprete de interfaz cooperan estrechamente para traducir combinaciones de teclas en acciones ejecutivas inmediatas. Esta sección examina la infraestructura interna que procesa estos eventos en el núcleo de Windows y desglosa las combinaciones más utilizadas en las versiones Windows 10 y Windows 11.

2.1. Procesamiento de mensajes y API en Win32k

La arquitectura de interfaz de Windows aloja sus subsistemas críticos (USER y GDI) en espacio de núcleo dentro de `win32k.sys` y `win32kbase.sys` para suprimir la sobrecarga por cambios de contexto entre modos de ejecución (Deitel et al., 2004, §21.4, p. 1043). El flujo de teclado es administrado por el hilo de entrada cruda (RIT), que extrae paquetes de `kbdclass.sys` y gestiona los estados de teclas modificadoras (Microsoft Corporation, 2023a).

Al recibir una pulsación, el RIT aplica una estricta jerarquía de intercepción: primero evalúa la secuencia de atención segura (SAS, `Ctrl + Alt + Del`), desviándola exclusivamente al proceso `winlogon.exe` en el escritorio seguro para impedir la captura maliciosa de credenciales; en segundo término, verifica las combinaciones globales dadas de alta con `RegisterHotKey`, despachando un mensaje `WM_HOTKEY` prioritario al hilo registrado; y finalmente, si no hay intercepción, deposita los eventos ordinarios (`WM_KEYDOWN`, `WM_CHAR`) en la cola de la ventana con foco de entrada (Microsoft Corporation, 2023b).

2.2. Catálogo técnico de combinaciones esenciales

Las combinaciones de teclas más utilizadas en Windows 10 y Windows 11 se articulan en torno a la administración del almacenamiento, el control de procesos, la composición espacial de ventanas y la captura gráfica (Microsoft Corporation, 2024).

En la exploración y administración de archivos, la combinación más representativa es `Win + E`, que invoca una nueva instancia de `explorer.exe` en la vista de acceso rápido o en la raíz local. Dentro del explorador, `Alt + Flecha Arriba` asciende en el árbol jerárquico, `Alt + Flecha Izquierda` conmuta al directorio previo en el historial, `F2` edita el nombre del nodo seleccionado y `Ctrl + Shift + N` crea inmediatamente una nueva carpeta en el CWD. Para el control del sistema, `Ctrl + Shift + Esc` invoca directamente el Administrador de Tareas (`taskmgr.exe`) con prioridad ejecutiva elevada, permitiendo diagnosticar y forzar la terminación de procesos aun si el shell gráfico está bloqueado. Asimismo, `Win + R` abre el cuadro de ejecución (`RunFileDlg` en `shell32.dll`) para binarios y consolas administrativas resueltas mediante `PATH`, `Win + X` despliega el Menú de Enlace Rápido (*Power User Menu*) y `Win + I` abre la Configuración mediante el protocolo `ms-settings:`.

En la gestión de ventanas y superficies, el Administrador de Ventanas del Escritorio (DWM) coordina la manipulación espacial. El atajo `Alt + Tab` conmuta entre aplicaciones activas alterando su orden de profundidad (*z-order*), mientras que `Win + Tab` activa la Vista de Tareas. La creación de nuevos escritorios virtuales se realiza con `Win + Ctrl + D`, la navegación horizontal entre ellos con `Win + Ctrl + Flecha Izquierda / Derecha`, y su desasignación con `Win + Ctrl + F4`. Para la organización en mosaico, `Win + Flechas de dirección` calcula el rectángulo delimitador mediante la API `SetWindowPos` (*Snap Assist*), `Win + D` minimiza o restaura en bloque las ventanas visibles, y `Win + L` invoca `LockWorkStation` en `user32.dll` transfiriendo la pantalla al entorno seguro de autenticación.

Finalmente, en el dominio de captura gráfica y portapapeles, `Win + Shift + S` invoca la herramienta de recortes (`SnippingTool.exe`), congelando el fotograma del DWM y depositando la región seleccionada en el portapapeles en formatos DIB y PNG (Microsoft Corporation, 2024). Esta función se complementa con `Win + V`, que abre el historial del portapapeles administrado por el servicio *Clipboard Broker Service*. Para capturas directas, `Impr Pant` copia la pantalla completa y `Alt + Impr Pant` restringe la muestra a la geometría de la ventana con foco.

La Tabla 2 sintetiza las principales combinaciones de teclado en Microsoft Windows, vinculando cada atajo con el componente del sistema operativo que lo procesa y su impacto técnico en la ejecución del entorno.

3. Atajos de teclado en entornos GNU/Linux

En los sistemas operativos basados en el núcleo Linux, la arquitectura gráfica y el procesamiento de eventos de teclado se fundamentan en una separación modular entre el kernel, el servidor o compositor gráfico y el entorno de escritorio. Como sistema de referencia formal para el presente análisis, se toma una distribución GNU/Linux moderna ejecutando el entorno GNOME (versión 40 en adelante) sobre el compositor Mutter bajo el protocolo Wayland, contrastándolo con la arquitectura clásica de X11.

3.1. Servidores gráficos: X11 y compositores Wayland

En la arquitectura histórica de X11, el servidor centralizado Xorg capturaba atajos globales mediante la función `XGrabKey`, desviando eventos al cliente solicitante. No obstante, la ausencia de aislamiento permitía que cualquier proceso leyera arbitrariamente el flujo global del teclado (*keylogging*).

Tabla 2: Catálogo técnico de atajos de teclado esenciales en Microsoft Windows.

Atajo de teclado	**Componente / Proceso**	**Mecanismo y acción técnica en el SO**
`Win + E`	Shell (`explorer.exe`)	Instancia una nueva ventana del Explorador de archivos sobre el espacio de nombres virtual del sistema.
`Ctrl + Shift + Esc`	Núcleo / `taskmgr.exe`	Invocación prioritaria del Administrador de Tareas para monitoreo de hilos y terminación de procesos.
`Win + R`	`shell32.dll` (`RunFileDlg`)	Abre el cuadro de ejecución que resuelve rutas de binarios y consolas administrativas en el `PATH`.
`Win + X`	Menú del sistema (*Quick Link*)	Despliega accesos directos administrativos predefinidos (Terminal elevada, Administrador de discos).
`Win + L`	`user32.dll` (`LockWorkStation`)	Bloquea la sesión de usuario y conmuta la pantalla al escritorio seguro gestionado por `winlogon.exe`.
`Win + D`	Gestor de ventanas / DWM	Conmuta el estado de visibilidad del escritorio, minimizando o restaurando todas las ventanas activas.
`Alt + Tab`	Compositor DWM	Alterna el foco del teclado y reordena la pila de ventanas según su jerarquía de profundidad (*z-order*).
`Win + Tab`	Vista de tareas / DWM	Presenta la vista panorámica de ventanas y paneles de administración de escritorios virtuales.
`Win + Ctrl + D`	Gestor de sesiones / DWM	Crea e inicializa dinámicamente una nueva superficie de escritorio virtual en la sesión interactiva.
`Win + Flechas`	API Win32 (`SetWindowPos`)	Asistente de anclaje (*Snap Assist*): redimensiona y ajusta la ventana en mitades o cuadrantes.
`Win + Shift + S`	Herramienta de recortes	Congela el fotograma del DWM y exporta la región seleccionada al portapapeles en formato DIB/PNG.
`Win + V`	*Clipboard Broker Service*	Despliega el historial persistente de elementos almacenados en el búfer múltiple del portapapeles.

Bajo Wayland, el servidor gráfico, el gestor de ventanas y el compositor se unifican en un único proceso (Mutter en GNOME) que interactúa directamente con el núcleo vía KMS/DRM y `libinput`. Wayland impone un aislamiento estricto de superficies: cada aplicación solo recibe eventos cuando posee el foco activo. Los atajos globales del sistema son evaluados y consumidos internamente por el compositor, mientras que las aplicaciones enjauladas (Flatpak) deben solicitar atajos mediante el portal XDG `org.freedesktop.portal.GlobalShortcuts` sobre D-Bus, requiriendo autorización explícita del usuario (Flatpak and Freedesktop.org Contributors, 2024a).

3.2. Catálogo de atajos en entorno GNOME sobre Wayland

El entorno de escritorio GNOME provee un catálogo estandarizado de atajos procesados por `gnome-settings-daemon` y el compositor Mutter (The GNOME Project, 2024). La invocación del intérprete de comandos mediante `Ctrl + Alt + T` despacha la terminal predeterminada instanciando un proceso hijo vinculado a un pseudoterminal (*pty*) mediante las llamadas `fork()` y `execve()`. Para el lanzamiento de aplicaciones y supervisión, la tecla `Super` activa la Vista General de Actividades, exponiendo un mosaico de ventanas junto a la gestión de espacios de trabajo dinámicos, navegables horizontalmente con `Super + Re Pág / Av Pág`, mientras que `Alt + F2` abre el cuadro de ejecución directa.

La manipulación espacial de superficies se coordina mediante atajos del compositor: `Alt + Tab` conmuta entre aplicaciones reordenando el plano visual, `Alt + ‘` alterna entre ventanas de una misma aplicación (*window cycling*), y `Super + Flechas` ajusta ventanas en mitades simétricas o maximización. Para seguridad y control de sesión, `Super + L` transfiere el control a GDM mediante D-Bus bloqueando la pantalla, y `Ctrl + Alt + Supr` despliega el diálogo de apagado. En el nivel más bajo del núcleo, las combinaciones de emergencia `Ctrl + Alt + F1-F6` conmutan fuera del servidor gráfico hacia consolas virtuales de texto puro (TTY) administradas por `systemd-vconsole-setup` (Carretero Pérez et al., 2001, §7.8, p. 399).

3.3. Mecanismos y herramientas de captura de pantalla

La captura de pantalla en GNU/Linux bajo compositores modernos constituye un caso ejemplar de interacción entre atajos de teclado y mecanismos de seguridad del sistema operativo. Debido a que el protocolo Wayland prohíbe que cualquier cliente de usuario lea directamente los búferes de fotogramas del servidor, la captura debe ser efectuada directamente por el compositor o delegada mediante protocolos seguros.

En el entorno GNOME (a partir de la versión 42), la pulsación de `PrintScreen` (o `Impr Pant`) despliega una interfaz interactiva superpuesta (*screenshot overlay*) integrada en Mutter y GNOME Shell (The GNOME Project, 2024). Esta herramienta congela el fotograma y ofrece tres modalidades de selección: pantalla completa (abarca todos los monitores), ventana específica (delimita la geometría activa) o selección de área rectangular por coordenadas. Permite además alternar hacia la grabación de vídeo (*screencast*). Al confirmarse, el compositor exporta la imagen a PNG en `~/Imágenes/Capturas de pantalla/` y copia los píxeles al portapapeles. Para capturas directas sin menú emergente, `Shift + PrintScreen` captura la pantalla completa, `Alt + PrintScreen` captura la ventana enfocada, y `Shift + Ctrl + Alt + R` activa el *screencast* en tiempo real.

En aplicaciones enjauladas (Flatpak) o herramientas externas, la operación se canaliza a través del portal de escritorio XDG mediante org.freedesktop.portal.Screenshot so-

bre D-Bus (Flatpak and Freedesktop.org Contributors, 2024b). El compositor Mutter solicita confirmación al usuario y deposita el archivo en un sistema de archivos virtual seguro (`/run/user/$UID/doc/`), impidiendo capturas silenciosas no autorizadas. En entornos de línea de comandos, destacan utilidades como `gnome-screenshot`, `grim` (vía el protocolo Wayland `wlr-screencopy-unstable-v1`) y `slurp` para la selección interactiva de geometrías.

La Tabla 3 resume las combinaciones de teclas más representativas en GNU/Linux bajo GNOME y Wayland, su componente de procesamiento y su repercusión técnica en el sistema operativo.

Tabla 3: Catálogo técnico de atajos esenciales en GNU/Linux (GNOME sobre Wayland).

Atajo de teclado	**Componente / Servicio**	**Mecanismo y acción técnica en el SO**
`Ctrl + Alt + T`	`gnome-settings-daemon`	Invoca la terminal predeterminada (`fork` y `execve` asociados a un nuevo *pty*).
`PrintScreen`	GNOME Shell / Mutter	Despliega la interfaz interactiva de captura (pantalla, ventana o área) y *screencast*.
`Shift + PrintScreen`	Compositor Mutter	Captura directa de la pantalla completa y almacenamiento atómico en formato PNG.
`Alt + PrintScreen`	Compositor Mutter	Captura exclusiva de la superficie geométrica de la ventana con foco activo.
`Super`	GNOME Shell	Conmuta a la Vista General de Actividades, exponiendo mosaico de ventanas y escritorios.
`Super + Re Pág / Av Pág`	Gestor de espacios dinámicos	Navegación horizontal entre escritorios virtuales asignados dinámicamente en memoria.
`Alt + Tab`	Compositor Mutter	Conmuta el foco de entrada entre aplicaciones activas alterando la jerarquía visual.
`Alt + ‘`	GNOME Shell	Conmutación interna de ventanas (*window cycling*) entre instancias de una misma aplicación.
`Super + Flechas`	Compositor Mutter	Ajuste geométrico de superficies: maximización vertical o división en mitades laterales.
`Super + L`	GDM (`org.gnome.ScreenSaver`)	Bloquea la sesión y transfiere la salida gráfica al protector de pantalla autenticado.
`Ctrl + Alt + F1-F6`	Núcleo Linux / TTY	Conmuta fuera del servidor gráfico hacia consolas virtuales de texto puro del sistema.

4. Arquitectura y modelo modal del editor Vi

En los sistemas operativos tipo UNIX y las distribuciones GNU/Linux, el editor de texto es una herramienta crítica para la administración remota mediante SSH y consolas de rescate. En este contexto, `vi` y su versión extendida Vim (*Vi IMproved*) representan el estándar de edición modal interactiva en interfaces textuales (*text user interface*, TUI).

4.1. Evolución histórica: de ex a POSIX.1 y Vim

La génesis de `vi` data de 1976, cuando Bill Joy diseñó el editor extendido `ex` para BSD UNIX a partir del clásico editor de líneas `ed`, introduciendo un modo visual de pantalla completa (`vi`). El diseño ergonómico estuvo determinado por la terminal Lear Siegler ADM-3A, cuya carencia de teclas de cursor motivó el uso de las teclas alfanuméricas `h`, `j`, `k` y `l` para el desplazamiento bidimensional, y cuya tecla *home* originó la tilde (`~`) para el directorio del usuario.

En 1992, la norma IEEE Std 1003.2 (consolidada en POSIX.1, IEEE Std 1003.1) estandarizó a `vi` como la utilidad de pantalla obligatoria para sistemas compatibles con UNIX (IEEE and The Open Group, 2018). Posteriormente, Bram Moolenaar presentó en 1991 la primera versión de Vim, introduciendo resaltado sintáctico, árbol de deshacer multinivel, ventanas divididas y selección visual estructurada (Moolenaar & The Vim Development Group, 2024; Oualline, 2001).

4.2. Máquina de estados finitos y modos de ejecución

A diferencia de los editores convencionales, Vi opera como una máquina de estados finitos (FSM) determinista, donde la semántica de cada tecla depende del modo activo: Normal (navegación y comandos sin modificadores continuos), Inserción (captura de texto en el búfer), Línea o Ex (directivas globales, entrada/salida y configuración), Visual (selección matricial de texto) y Reemplazo (sobrescritura secuencial). Para garantizar la tolerancia a fallos, el editor mantiene el texto en memoria RAM (búfer) y sincroniza transaccionalmente las páginas sucias en un archivo de intercambio oculto (`.swp`) en disco, habilitando la recuperación ante caídas del sistema (`vim -r`) y funcionando como cerrojo (*lockfile*) ante accesos concurrentes (Oualline, 2001). La Tabla 4 resume estos modos operativos y sus transiciones.

5. Operativa, gramática de comandos y flujo en Vi

El editor Vi formula cada instrucción siguiendo una gramática composicional formal basada en verbos y sustantivos:

$$\text{Comando} = [\text{conteo}] + \text{operador} + [\text{conteo}] + \text{movimiento / objeto de texto} \qquad (1)$$

5.1. Composición sintáctica: operadores, conteos y movimientos

Los operadores actúan como verbos ejecutivos en modo Normal: `d` (borrar/cortar), `c` (cambiar e ingresar a Inserción), `y` (copiar hacia registro), > / < (sangría) y = (reindentar). Al

Tabla 4: Estados de la máquina modal en el editor Vi/Vim.

Modo	Activación y retorno	Semántica y función en el SO
Normal	Estado inicial; retorno con `<Esc>` o `Ctrl + [`.	Comandos de navegación y operadores léxicos en la fila central del teclado.
Inserción	Operadores `i`, `I`, `a`, `A`, `o`, `O`, `s`.	Captura texto en el búfer en RAM; sincroniza periódicamente con el `.swp`.
Línea (Ex)	Caracteres `:`, `/`, `?`; salida con `<Enter>`.	Interpreta órdenes en la última línea: I/O en disco (`:w`), salida y comandos de shell.
Visual	Teclas `v` (carácter), `V` (línea), `Ctrl + v` (bloque).	Delimita regiones geométricas en pantalla para transformaciones atómicas.
Reemplazo	Tecla `R`; retorno con `<Esc>`.	Sobrescribe los caracteres del búfer preservando la alineación visual circundante.

duplicarse (`dd`, `yy`), operan sobre la línea actual, y al prefijarse con un número (`3dd`), repiten la acción linealmente (Oualline, 2001, §2, p. 35).

Los movimientos delimitan el alcance del operador o desplazan el cursor: caracteres y líneas (`hjkl`, `0`, `^`, `$`), palabras (`w`, `b`, `e`), búsquedas en línea (`f`, `F`, `t`, `T`), estructuras sintácticas (`{ }`, `( )`, `%` para pares) y saltos globales (`gg`, `G`). Asimismo, los objetos de texto (*text objects*) permiten manipular entidades sintácticas internas (`iw`, `i"`, `i(`, `i{`) o completas (`aw`, `a"`, `a{`) desde cualquier punto en su interior (Moolenaar & The Vim Development Group, 2024).

5.2. Manipulación de búferes, registros y sustitución

Vi desacopla la persistencia en disco de las estructuras de edición en memoria RAM (búferes), organizables en ventanas divididas (`:split`, `:vsplit`) y pestañas. Para transferencias de datos, implementa una matriz de registros desacoplada: el registro no nombrado (`""`), 26 registros nombrados (`"a`–`"z`, admitiendo acumulación con mayúsculas), el registro de copia (`"0`), el registro nulo (`"_`) y los registros del portapapeles del sistema operativo (`"+` y `"*`) (Moolenaar & The Vim Development Group, 2024).

La sustitución global se ejecuta en modo Ex con la directiva `s`:

$$:[rango]\mathsf{s}/patrón/reemplazo/[banderas] \tag{2}$$

Donde `%` abarca todo el archivo, `g` aplica el reemplazo a todas las ocurrencias por línea y `c` solicita confirmación interactiva. La Tabla 5 sintetiza esta gramática.

6. Demostración práctica de edición en Vim

Para consolidar los principios del modelo modal y la gramática formal, se desarrolla una sesión práctica de edición de código en C empleando Vim nativo en GNU/Linux. El ejercicio documenta el ciclo de vida del archivo: inicialización y modo Normal, captura en modo Inserción, transformaciones estructurales, manipulación matricial de columnas y persistencia en disco mediante el modo Ex.

Tabla 5: Sintaxis de operadores, movimientos y registros en Vi/Vim.

Categoría	Instrucciones representativas	Función y semántica operativa
Operadores nucleares	`d` (borrar), `c` (cambiar), `y` (copiar), `>` / `<` (sangría), `=` (reindentar).	Verbos ejecutivos; actúan sobre un movimiento o línea completa si se duplican (`dd`).
Movimientos	`w`, `b`, `e` (palabra); `0`, `^`, `$` (línea); `f`, `t` (búsqueda); `gg`, `G` (archivo).	Delimitan la extensión geométrica de la orden o trasladan el cursor en el búfer.
Objetos de texto	`iw`, `aw` (palabra); `i"`, `a"` (comillas); `i(`, `i{` (bloques de código).	Permiten manipular bloques sintácticos internos o con delimitadores de forma directa.
Registros de memoria	`""` (defecto), `"a-z` (nombrados), `"0` (copia), `"+` (portapapeles SO), `"_` (nulo).	Matriz de memoria persistente durante la sesión; soporta acumulación y portapapeles del SO.
Sustitución regex	`:%s/origen/destino/g`, `:%s/patrón/reemplazo/gc`.	Reemplazo global con expresiones regulares y confirmación interactiva en modo Ex.

6.1. Inicialización de sesión y modo Normal

La sesión inicia ejecutando `vim ejemplo_so.c`. El proceso adquiere descriptores para `stdin`, `stdout` y `stderr`, conmuta el controlador TTY a modo crudo (*raw mode*) desactivando el eco local, e inicializa en disco el archivo de intercambio `.ejemplo_so.c.swp` para tolerancia a fallos. Vim sitúa al usuario en modo Normal en la coordenada $(1, 1)$, donde las tildes (~) indican líneas fuera del búfer de memoria y la línea de estado confirma la creación de un nuevo búfer vacío (Oualline, 2001, §2, p. 28), como se ilustra en la Img. 1.



Imagen 1: Inicialización de sesión en Vim y estado por defecto en modo Normal (Oualline, 2001, §2, p. 28).

6.2. Captura y modificación en modo Inserción

Al pulsar la tecla `i`, el editor conmuta al modo Inserción, emitiendo secuencias ANSI para transformar el cursor en una línea vertical e indicando `- INSERT -` en la barra de estado. En este estado se redacta un programa en C enfocado en llamadas al sistema para gestión de procesos (`fork()`, `getpid()` y `wait()`). Al concluir, la tecla `<Esc>` (o `Ctrl + [`) retorna al modo Normal, sincronizando páginas sucias con el archivo `.swp` en almacenamiento (Oualline, 2001, §2, p. 32), como muestra la Img. 2.



Imagen 2: Ingreso interactivo de código fuente en modo Inserción con indicador de estado (Oualline, 2001, §2, p. 32).

6.3. Operaciones estructurales en modo Normal

En modo Normal, las modificaciones se rigen por la gramática de la Ecuación (1). La instrucción `yy` copia la línea del cursor al registro anónimo, y `p` la inserta en la posición inferior, mientras que `dd` suprime sentencias completas y `cw` sustituye palabras clave. El operador punto (`.`) reejecuta la última acción atómica, y el historial de cambios permite deshacer con `u` o rehacer con `Ctrl + r` (Oualline, 2001, §2, p. 36). La Img. 3 documenta la duplicación y ajuste de variables del proceso.

6.4. Selección por bloques en modo Visual

El modo Visual provee selección estructurada en variantes por carácter (`v`), línea (`V`) o bloque matricial (`Ctrl + v`). Mediante el modo bloque visual, el operador delimita una columna rectangular vertical con las teclas `j` o `k` sobre múltiples líneas de código. Al pulsar `I`, redactar los caracteres de comentario (`// `) y presionar `<Esc>`, Vim propaga instantáneamente la modificación a todas las líneas del bloque seleccionado (Oualline, 2001, §4, p. 85), como se aprecia en la Img. 4.



Imagen 3: Aplicación de operadores gramaticales y movimientos estructurales en modo Normal (Oualline, 2001, §2, p. 36).



Imagen 4: Selección matricial de columnas en modo Visual Bloque para edición concurrente (Oualline, 2001, §4, p. 85).

6.5. Comandos de sustitución y persistencia en modo Ex

El modo Ex se invoca con el carácter `:`. La directiva `:%s/int/pid_t/gc` analiza el búfer completo mediante expresiones regulares, solicitando confirmación interactiva (`c`) para sustituir el tipo primitivo por el identificador del kernel `pid_t`. Finalmente, la instrucción `:wq` sincroniza el búfer con el inodo en disco mediante `write()` y `fsync()`, elimina el archivo `.swp` y restaura la terminal al modo canónico del sistema operativo (IEEE and The Open Group, 2018), tal como se expone en la Img. 5.

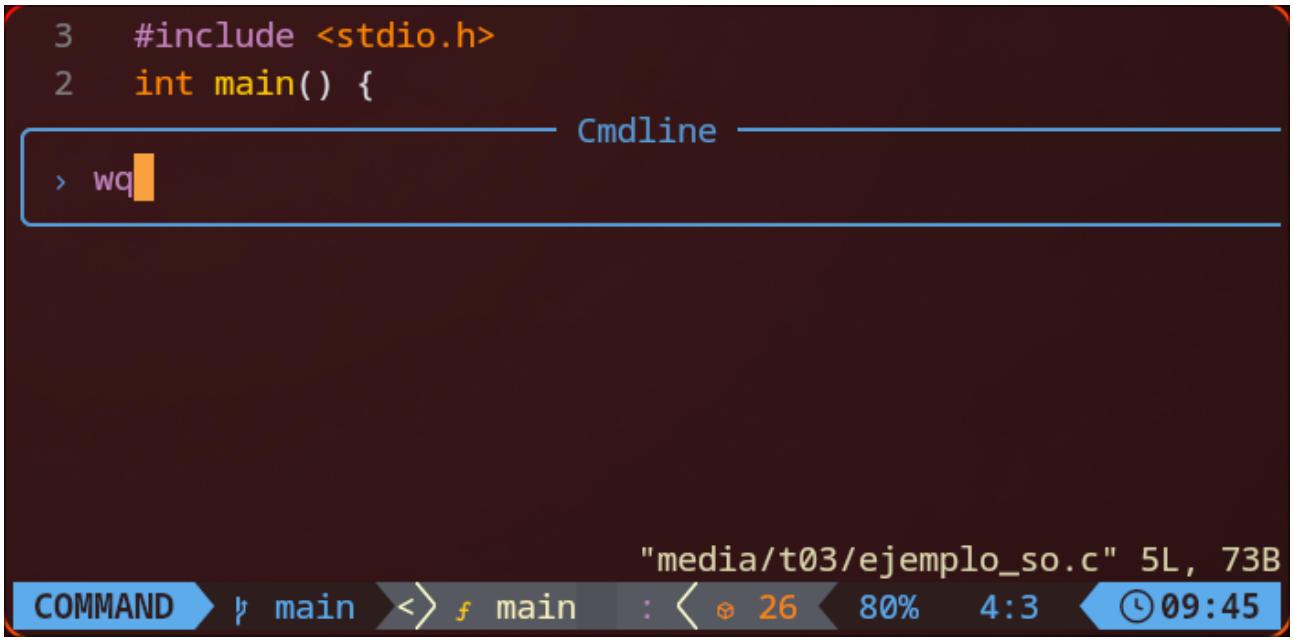


Imagen 5: Ejecución de directivas globales en modo Ex y persistencia en disco con `:wq` (IEEE and The Open Group, 2018).

7. Estructura de directorios: rutas absolutas y relativas

El sistema de archivos proporciona el mecanismo fundamental para la persistencia organizada, denominación y recuperación de la información en almacenamiento secundario. Para que el usuario y los procesos del sistema interactúen con los archivos sin requerir el direccionamiento físico por sectores o bloques, los sistemas operativos articulan una estructura jerárquica de directorios en forma de árbol invertido (Stallings, 2018, §12.2, p. 535). Esta sección analiza la gestión del contexto de trabajo y formaliza la distinción operativa entre rutas absolutas y relativas en núcleos POSIX y Windows NT.

7.1. Jerarquía del sistema de archivos y directorio de trabajo

En la arquitectura de sistemas tipo UNIX y GNU/Linux, el sistema de archivos virtual (*virtual file system*, VFS) unifica todos los dispositivos de almacenamiento físico y sistemas lógicos en un único espacio de nombres jerárquico que desciende desde un nodo raíz global denominado `/` (*root*). Por el contrario, Microsoft Windows NT estructura su espacio accesible mediante letras de unidad independientes asignadas a volúmenes lógicos (`C:\`, `D:\`), las

cuales son gestionadas internamente por el Administrador de Objetos del núcleo como enlaces simbólicos hacia dispositivos nativos (como `\Device\HarddiskVolume1`) (Deitel et al., 2004, §21.6, p. 1058).

Para resolver nombres de archivo de forma contextual, el sistema operativo vincula a cada proceso activo un atributo denominado directorio de trabajo actual (*current working directory*, CWD). Esta propiedad reside en la estructura de control del proceso en memoria del núcleo: en Linux, dentro de la estructura `task_struct->fs->pwd` (compuesta por punteros al inodo y a la entrada de directorio o *dentry*), y en Windows, en los parámetros del proceso dentro del bloque de entorno del proceso (*process environment block*, PEB). Cuando un proceso inicia su ejecución, hereda el CWD de su proceso padre (mediante `fork()` o `CreateProcess()`), pudiendo alterarlo dinámicamente durante su ciclo de vida mediante la llamada al sistema `chdir()` en POSIX o `SetCurrentDirectory()` en la API Win32 (Carretero Pérez et al., 2001, §6.3, p. 309).

7.2. Definición formal de ruta absoluta y ruta relativa

La localización de cualquier entidad dentro del árbol jerárquico se expresa mediante una trayectoria o ruta (*pathname*), la cual consiste en una secuencia de identificadores textuales delimitados por separadores de directorio (`/` en UNIX y `\` en Windows) (Silberschatz et al., 2006, §10.3, p. 342):

1. **Ruta absoluta (*absolute path*):** Es una trayectoria canónica e invariable que especifica la ubicación de un archivo o directorio comenzando inequívocamente desde el nodo raíz del sistema de archivos (`/` en entornos POSIX o la letra de unidad seguida del delimitador de raíz `C:\` en Windows). Su resolución es completamente autónoma e independiente del CWD del proceso emisor: dos procesos con directorios de trabajo dispares que invoquen una misma ruta absoluta (ej. `/etc/passwd` o `C:\Windows\System32\drivers`) accederán idénticamente al mismo nodo físico en el almacenamiento secundario.
2. **Ruta relativa (*relative path*):** Es una trayectoria contextual que no inicia con el delimitador de raíz, estableciendo su origen de búsqueda a partir del CWD del proceso que la invoca. Su interpretación depende enteramente del estado dinámico del proceso: la referencia `config/datos.txt` se resuelve como `/home/usuario/config/datos.txt` si el CWD es `/home/usuario`, pero fallará o apuntará a otro inodo si el proceso migra su contexto a `/var/log`.

Para posibilitar la navegación ascendente y la referencia local en rutas relativas, los sistemas de archivos incorporan de manera obligatoria en cada directorio dos entradas especiales: el punto (`.`), que constituye un enlace duro reflexivo hacia el propio directorio actual, y el doble punto (`..`), que apunta directamente al inodo del directorio padre en la jerarquía (Silberschatz et al., 2006, §10.3, p. 343). De este modo, expresiones como `../../bin/script.sh` ascienden dos niveles jerárquicos desde el CWD antes de descender al nodo objetivo.

7.3. Mecanismos de resolución en núcleos POSIX y Windows NT

En el espacio del núcleo, la conversión de una ruta textual en una referencia a inodo o manejador de archivo (*handle*) exige un proceso iterativo de resolución componente por componente denominado recorrido de trayectoria (*path walking*).

En sistemas GNU/Linux, el núcleo ejecuta la rutina interna `path_walk()` del VFS. Si la ruta inicia con `/`, la resolución arranca desde el inodo raíz apuntado por `current->fs->root`; si no, inicia en `current->fs->pwd`. Para minimizar los costosos accesos de lectura a disco, el núcleo consulta la memoria caché de entradas de directorio (*dentry cache* o *dcache*), verificando secuencialmente los permisos de ejecución (`x`) necesarios para atravesar cada subdirectorio intermedio. En entornos modernos, las llamadas al sistema clásicas basadas en rutas globales (como `open()`) han sido complementadas por variantes relativas orientadas a descriptores de archivo, tales como `openat()` y `unlinkat()`. Estas funciones reciben un descriptor abierto de directorio (*dirfd*) junto a una ruta relativa, permitiendo al proceso fijar atómicamente el contexto de resolución y mitigando condiciones de carrera de verificación frente a uso (*time-of-check to time-of-use*, TOCTOU) en software multi-hilo (Carretero Pérez et al., 2001).

En Windows NT, las llamadas de la API Win32 (`CreateFile`) se canalizan hacia la llamada nativa `NtCreateFile`. El Administrador de Objetos del núcleo antepone el espacio de nombres de dispositivos de usuario (`\??\C:\...`), normaliza barras inclinadas y consulta las estructuras del sistema de archivos NTFS para resolver entradas de índice en la tabla maestra de archivos (*master file table*, MFT) o procesar puntos de reanálisis (*reparse points*) asociados a enlaces simbólicos y uniones de directorios.

La Tabla 6 sintetiza las diferencias técnicas y de diseño entre rutas absolutas y relativas.

Tabla 6: Comparativa técnica y funcional entre rutas absolutas y relativas.

Criterio técnico	Ruta absoluta	Ruta relativa
Origen sintáctico	Inicia estrictamente en la raíz (`/` en UNIX; `<unidad>:\` en Windows).	Inicia sin separador de raíz; referenciada al contexto actual (`./`, `../` o nombre directo).
Dependencia del CWD	Invariante; no se altera ante modificaciones en el estado del proceso ejecutor.	Dependiente; la resolución final cambia si el proceso modifica su CWD con `chdir()`.
Resolución en el SO	Recorrido jerárquico desde `fs->root` o volumen lógico asignado.	Recorrido a partir de `fs->pwd` o descriptor de directorio relativo (`dirfd` en `openat`).
Portabilidad	Baja; acoplada a la disposición absoluta de directorios y volúmenes de un equipo específico.	Elevada; permite empaquetar árboles de proyectos y dependencias autocontenidas.
Seguridad de concurrencia	Susceptible a sustitución encubierta de rutas en entornos multiusuario.	Permite acceso atómico mediante descriptores seguros (`openat()`) frente a TOCTOU.

8. Comandos de terminal en sistemas GNU/Linux

La interfaz de línea de comandos (*command-line interface*, CLI) en sistemas operativos GNU/Linux constituye el entorno primordial para la administración del sistema, la automatización mediante guiones (*shell scripts*) y el control de procesos. Lejos de ser un mero intermediario textual, el shell actúa como un potente lenguaje de programación interpretado que se comunica de forma directa con los servicios ejecutivos del núcleo mediante llamadas al sistema normalizadas por el estándar POSIX (Silberschatz et al., 2006, §2.2, p. 48). Esta sección analiza la arquitectura de ejecución de comandos y detalla diez de las utilidades de terminal más representativas del entorno.

8.1. Arquitectura del shell y ejecución de utilidades

El shell (predominantemente GNU Bash o Zsh en distribuciones contemporáneas) opera en espacio de usuario siguiendo un bucle clásico de lectura, evaluación, impresión y repetición (*read-eval-print loop*, REPL). Al recibir una línea de comandos, el analizador léxico segmenta la cadena en componentes léxicos (*tokens*), procesa expansiones de variables de entorno y comodines (*globbing*), y clasifica la instrucción en una de dos categorías estructurales: comandos internos (*built-ins*) y ejecutables externos (Carretero Pérez et al., 2001, §2.4, p. 48).

Los comandos internos (como `cd`, `pwd` o `exit`) se ejecutan directamente dentro del espacio de memoria del propio proceso del shell. Esta integración es indispensable para órdenes que alteran el estado interno del proceso, como modificar el directorio de trabajo activo (CWD) o definir variables de entorno, ya que un proceso hijo aislado no puede mutar los descriptores ni la estructura `task_struct` de su proceso padre. Por el contrario, los comandos externos corresponden a archivos binarios ubicados en directorios del sistema (como `/bin`, `/usr/bin` o `/usr/local/bin`), localizados a través de la variable de entorno `PATH`. Para ejecutarlos, el shell invoca la llamada al sistema `fork()` para clonar su hilo de ejecución y `execve()` para reemplazar la imagen del proceso hijo con el nuevo binario, coordinando la sincronización y el estado de salida mediante `waitpid()` (Stallings, 2018, §3.2, p. 120).

Asimismo, el shell administra los tres descriptores de archivo estándar asignados por defecto a cada proceso: entrada estándar (`stdin`, descriptor 0), salida estándar (`stdout`, descriptor 1) y error estándar (`stderr`, descriptor 2). Mediante llamadas `dup2()` y `pipe()`, el shell posibilita la redirección de flujos hacia archivos en disco (>, », <) y la interconexión concurrente de comandos mediante tuberías (*pipelines*, |), permitiendo canalizar la salida de un proceso directamente hacia la entrada del siguiente en memoria sin recurrir a almacenamiento secundario intermedio.

8.2. Catálogo de diez comandos fundamentales

Para interactuar eficazmente con el subsistema de archivos, la administración de memoria y el control de tareas en GNU/Linux, existe un conjunto de diez utilidades fundamentales cuya operatividad técnica se desglosa a continuación:

1. `ls` **(*list*):** Enumera las entradas contenidas en un directorio. En el núcleo, invoca la llamada al sistema `getdents64()` para leer bloques de entradas de directorio y consulta metadatos de inodos con `statx()` (o `lstat()` para enlaces simbólicos). Opciones como `-l` generan el listado largo (permisos POSIX, propietario, grupo, tamaño y marca temporal), `-a` expone archivos ocultos (cuyo nombre inicia con punto) y `-h` humaniza el tamaño en potencias de 1024.
2. `cd` **(*change directory*):** Modifica el directorio de trabajo del proceso del shell. Al ser un comando interno, invoca directamente la llamada `chdir()`, actualizando el puntero `fs->pwd` en la estructura del kernel. La invocación sin argumentos o con ~ traslada al directorio de inicio del usuario (`$HOME`), mientras que `cd -` retorna al directorio previo (`$OLDPWD`).
3. `pwd` **(*print working directory*):** Imprime la ruta absoluta del CWD activo. Invoca la llamada al sistema `getcwd()`, la cual inspecciona la tabla de dentries en el VFS para reconstruir la cadena canónica desde la raíz hasta el nodo actual. La bandera `-P` resuelve enlaces simbólicos mostrando la ruta física real.

4. `mkdir` **(*make directory*):** Crea nuevos nodos de tipo directorio en el sistema de archivos. Ejecuta la llamada al sistema `mkdir()` o `mkdirat()`, asignando un nuevo inodo con los bits de directorio habilitados y aplicando la máscara de modo de creación de archivos (`umask`). Con la opción `-p` (*parents*), crea recursivamente toda la jerarquía de subdirectorios faltantes sin arrojar errores si ya existen.
5. `cp` **(*copy*):** Duplica archivos o árboles de directorios. En núcleos Linux contemporáneos, utiliza llamadas optimizadas como `copy_file_range()` o `sendfile()`, permitiendo clonaciones directas entre descriptores en el kernel o reflejos instantáneos a nivel de bloques en sistemas de archivos con copia en escritura (*copy-on-write*, CoW) como Btrfs. La bandera `-r` aplica copia recursiva y `-p` preserva marcas temporales y permisos originales.
6. `mv` **(*move*):** Reubica o renombra nodos en la jerarquía. Si el destino reside en el mismo sistema de archivos, ejecuta la llamada atómica `rename()` o `renameat2()`, actualizando las entradas de directorio en disco sin copiar datos físicos. Si involucra diferentes dispositivos montados, recurre a una copia profunda seguida de la supresión del origen.
7. `rm` **(*remove*):** Elimina enlaces de archivos y directorios. En archivos regulares ejecuta `unlink()`, decrementando el contador de enlaces duros (*hard links*) del inodo; cuando este llega a cero y ningún proceso retiene descriptores abiertos sobre él, el núcleo libera los bloques de datos asociados en disco. La opción `-r` efectúa borrado recursivo en árboles de directorios y `-f` fuerza la operación suprimiendo solicitudes de confirmación.
8. `cat` **(*concatenate*):** Lee flujos secuenciales de datos desde archivos o `stdin` y los vuelca hacia `stdout`. Implementa bucles de lectura y escritura en memoria mediante `read()` y `write()`, resultando idóneo para inspeccionar archivos de configuración cortos o concatenar segmentos de texto mediante tuberías. Opciones como `-n` numeran las líneas emitidas.
9. `grep` **(*global regular expression print*):** Analiza flujos de texto en busca de patrones que coincidan con una expresión regular. Utiliza algoritmos de concordancia de patrones (como Boyer-Moore y autómatas finitos deterministas) procesando búferes intermedios con mínima latencia. Banderas como `-i` ignoran mayúsculas y minúsculas, `-r` realiza búsqueda recursiva en árboles y `-v` invierte el criterio filtrando líneas no coincidentes.
10. `ps` **(*process status*):** Reporta una instantánea de los procesos activos en ejecución. En GNU/Linux, no realiza sondeos destructivos; en su lugar, examina el sistema de archivos virtual `/proc`, donde el núcleo expone estructuras dinámicas de cada PID (`/proc/[pid]/stat`, `status`, `cmdline`). La sintaxis clásica `ps aux` o la normalizada POSIX `ps -ef` detallan el usuario emisor, consumo de CPU y memoria, estado de planificación y comando de invocación.

La Tabla 7 resume las características sintácticas, parámetros esenciales y llamadas al sistema asociadas a este catálogo de comandos.

9. Comandos del intérprete CMD en Windows

En el sistema operativo Microsoft Windows, el símbolo del sistema o intérprete de comandos (`cmd.exe`) representa el procesador de órdenes tradicional para la automatización administrativa mediante archivos por lotes (*batch files*, `.bat` y `.cmd`) y el control directo

Tabla 7: Catálogo técnico de diez comandos fundamentales en GNU/Linux.

Comando	Sintaxis representativa	Llamada al sistema	Función y semántica en el sistema operativo
`ls`	`ls -lah`	`getdents64`, `statx`	Lista contenidos de directorio recuperando metadatos y permisos desde inodos.
`cd`	`cd /var/log`	`chdir` (*builtin*)	Modifica el directorio de trabajo (CWD) del proceso en `task_struct->fs->pwd`.
`pwd`	`pwd -P`	`getcwd`	Reconstruye e imprime la ruta canónica absoluta hacia el CWD del proceso.
`mkdir`	`mkdir -p ruta/dir`	`mkdir`, `mkdirat`	Asigna un inodo de directorio y crea las entradas correspondientes en disco.
`cp`	`cp -rp orig dest`	`copy_file_range`	Duplica archivos o árboles preservando metadatos; optimizado en el kernel.
`mv`	`mv arch nuevo`	`rename`, `renameat2`	Reubica o renombra nodos atómicamente alterando enlaces de directorio.
`rm`	`rm -rf temp/`	`unlink`, `rmdir`	Decrementa enlaces duros del inodo y libera bloques de datos en almacenamiento.
`cat`	`cat -n arch.txt`	`read`, `write`	Vuelca secuencialmente flujos de bytes hacia el descriptor `stdout`.
`grep`	`grep -ri "patrón"`	Búferes VFS / regex	Filtra líneas por concordancia de expresiones regulares mediante autómatas.
`ps`	`ps aux` / `ps -ef`	Lectura en `/proc`	Inspecciona estructuras de procesos y estados de hilos expuestos por el kernel.

del sistema operativo. Aunque Windows incorpora entornos contemporáneos más complejos como PowerShell, `cmd.exe` permanece como un componente medular del subsistema Win32, caracterizado por una latencia de inicialización mínima, compatibilidad estricta con utilidades heredadas y acceso directo a las interfaces ejecutivas del núcleo Windows NT (Deitel et al., 2004, §21.4, p. 1045).

9.1. Arquitectura de cmd.exe y llamadas al sistema

A diferencia del shell de UNIX, que interactúa directamente con el controlador de pseudoterminal del núcleo, `cmd.exe` opera como un proceso Win32 desacoplado de la ventana física de visualización. Históricamente, la consola gráfica era gestionada por el proceso del subsistema cliente/servidor (`csrss.exe`); a partir de Windows 7, esta responsabilidad se transfirió al proceso de alojamiento de ventanas de consola (*Console Window Host*, `conhost.exe`) para mitigar vulnerabilidades de seguridad, y más recientemente a la infraestructura de pseudoconsolas (*ConPTY*) integrada en Windows Terminal.

Al procesar una línea de comandos, el analizador sintáctico de `cmd.exe` discrimina entre dos categorías de instrucciones: comandos internos y comandos externos. Los comandos internos se encuentran compilados directamente dentro del código ejecutable de `cmd.exe` (como `DIR`, `CD`, `MD`, `COPY`, `DEL` o `TYPE`). Su ejecución ocurre en el contexto del propio hilo del intérprete, lo cual es imprescindible para funciones que alteran el entorno del proceso (como modificar el CWD en el bloque PEB mediante `SetCurrentDirectoryW` o actualizar variables de entorno en la memoria local). Por su parte, los comandos externos corresponden a archivos ejecutables independientes (con extensiones `.exe`, `.com`) ubicados habitualmente en el directorio del sistema (`C:\Windows\System32`).

Para ejecutar una utilidad externa (como `tasklist.exe` o `robocopy.exe`), `cmd.exe` invoca la función de la API Win32 `CreateProcessW()`, la cual transfiere la solicitud a la llamada al sistema nativa del núcleo `NtCreateUserProcess()` (o `NtCreateProcessEx()`) (Silberschatz et al., 2006, §2.3, p. 52). Esta llamada crea el objeto de proceso y su hilo inicial, asigna el espacio de direcciones virtuales, enlaza los manejadores de entrada y salida estándar (`STD_INPUT_HANDLE`, `STD_OUTPUT_HANDLE`, `STD_ERROR_HANDLE`) y entrega los argumentos a través de la línea de comandos en el bloque PEB (Stallings, 2018, §3.2, p. 125).

9.2. Catálogo de diez comandos fundamentales

A continuación se analizan diez de los comandos más utilizados en `cmd.exe` en Windows 10 y Windows 11 para la administración de archivos, diagnóstico del almacenamiento y supervisión de procesos:

1. `dir` **(*directory*):** Lista los archivos y subdirectorios presentes en la ruta especificada. A nivel del núcleo, utiliza las funciones de la API Win32 `FindFirstFileW()` y `FindNextFileW()`, consultando los registros de índice de la tabla maestra de archivos (MFT) en particiones NTFS. Opciones como `/A` filtran por atributos específicos (archivos ocultos o de sistema), `/S` realiza una búsqueda recursiva a través de todos los subdirectorios y `/Q` expone la identidad del propietario del archivo.
2. `cd` / `chdir` **(*change directory*):** Muestra o modifica el directorio de trabajo del proceso. Al ser un comando interno, interactúa con la estructura PEB del intérprete invocando `SetCurrentDirectoryW()`. La inclusión del modificador `/D` resulta indispensable cuando la conmutación de contexto involucra cambiar simultáneamente de letra de unidad lógica (por ejemplo, conmutar de `C:\` a `D:\Proyectos`).

3. `md` / `mkdir` **(*make directory*):** Crea nuevos directorios en el sistema de archivos mediante la API `CreateDirectoryW()`. En las implementaciones modernas de `cmd.exe`, este comando incorpora una capacidad avanzada: si se le proporciona una ruta compuesta que contiene subdirectorios intermedios inexistentes (ej. `md config\logs\2026`), crea de forma atómica y transparente todos los niveles del árbol sin requerir parámetros adicionales.
4. `copy`**:** Duplica uno o varios archivos hacia otra ubicación de almacenamiento invocando `CopyFileW()` o manejando flujos de datos con `ReadFile()` y `WriteFile()`. Presenta la facultad técnica de concatenar múltiples archivos de texto plano o binarios en un único destino mediante el operador de adición (ej. `copy arch1.txt + arch2.txt destino.txt`), y permite verificar la integridad de la escritura mediante el parámetro `/V`.
5. `move`**:** Reubica archivos o renombra directorios completos en el árbol jerárquico. Si el origen y el destino residen dentro del mismo volumen lógico, ejecuta la función `MoveFileExW()` con la bandera `MOVEFILE_COPY_ALLOWED`, actualizando los punteros de nodo en la MFT sin duplicar los bloques de datos físicos. Si involucra volúmenes distintos, realiza una copia profunda seguida de la eliminación del archivo original.
6. `del` / `erase`**:** Suprime uno o varios archivos del almacenamiento secundario mediante `DeleteFileW()`, marcando los clústeres correspondientes como espacio disponible en el mapa de bits del sistema de archivos. Parámetros como `/F` fuerzan la eliminación de archivos con atributo de solo lectura, `/S` propaga la supresión recursivamente en subcarpetas y `/Q` activa el modo silencioso suprimiendo solicitudes interactivas de confirmación.
7. `type`**:** Despliega en la consola el contenido bruto de archivos de texto sin paginación (análogo funcional de `cat` en UNIX). Lee secuencialmente el archivo mediante `ReadFile()` y vuelca los bytes hacia el manejador de salida de la consola a través de `WriteConsoleW()` o `WriteFile()`, permitiendo redirigir su flujo hacia filtros mediante tuberías.
8. `tasklist`**:** Utilidad externa (`tasklist.exe`) que reporta el inventario completo de tareas y procesos activos. Consulta las estructuras del núcleo mediante la interfaz Toolhelp32 o llamadas nativas a `NtQuerySystemInformation()`, exponiendo el nombre de imagen, identificador de proceso (PID), nombre de sesión, número de sesión y el consumo de memoria en el conjunto de trabajo (*working set*). La opción `/SVC` detalla los servicios de Windows alojados en cada instancia de `svchost.exe`, y `/FI` aplica filtros avanzados de consulta.
9. `taskkill`**:** Herramienta externa (`taskkill.exe`) diseñada para finalizar procesos en ejecución. Localiza el proceso mediante `OpenProcess()` solicitando el privilegio de seguridad `PROCESS_TERMINATE` e invoca la llamada al sistema `TerminateProcess()`. Banderas esenciales incluyen `/PID <id>` o `/IM <imagen.exe>`, `/F` para imponer una terminación forzada no negociable y `/T` para finalizar recursivamente el proceso objetivo y todo su árbol de procesos secundarios subordinados.
10. `robocopy` **(*robust file copy*):** Utilidad avanzada de replicación y sincronización de estructuras de directorios (`robocopy.exe`). Diseñada para entornos empresariales tolerantes a fallos de red, soporta la reanudación automática de transferencias interrumpidas. Banderas críticas son `/E` (copia recursiva de subdirectorios, incluidos los vacíos), `/MIR` (modo espejo reflejo que sincroniza y purga destinos), `/SEC` (preserva listas de control de acceso NTFS / ACL y marcas de tiempo) y `/MT` (transferencia multi-hilo concurrente).

La Tabla 8 sintetiza las propiedades técnicas, tipo de implementación y funciones asociadas a este catálogo de comandos en Windows.

Tabla 8: Catálogo técnico de diez comandos fundamentales en CMD de Windows.

Comando	Sintaxis representativa	Tipo y API Win32	Función y semántica en el sistema operativo
`dir`	`dir /A /S /Q`	Interno / `FindNextFileW`	Lista contenidos de directorio inspeccionando la MFT en particiones NTFS.
`cd`	`cd /D D:\datos`	Interno / `SetCurrentDirectory`	Modifica el CWD en el bloque PEB, conmutando opcionalmente de letra de unidad.
`md`	`md ruta\sub\dir`	Interno / `CreateDirectoryW`	Crea carpetas físicas en disco, autogenerando niveles intermedios de jerarquía.
`copy`	`copy a.txt+b.txt c.txt`	Interno / `CopyFileW`	Duplica y permite concatenar archivos con verificación de integridad opcional.
`move`	`move orig dest`	Interno / `MoveFileExW`	Reubica o renombra nodos actualizando metadatos de directorio en la MFT.
`del`	`del /F /S /Q arch`	Interno / `DeleteFileW`	Marca sectores como libres y desvincula el nodo del sistema de archivos.
`type`	`type config.ini`	Interno / `WriteConsoleW`	Vuelca secuencialmente flujos de texto sin formato hacia el búfer de consola.
`tasklist`	`tasklist /SVC /FI ...`	Externo / `Toolhelp32`	Monitorea procesos activos, servicios asociados y consumo de Working Set.
`taskkill`	`taskkill /F /T /PID 12`	Externo / `TerminateProcess`	Envía señal de terminación forzada y recursiva a procesos y sus hilos hijos.
`robocopy`	`robocopy orig dest /MIR`	Externo / E/S multi-hilo	Replicación robusta de árboles jerárquicos con preservación de ACL y reanudación.

10. Análisis comparativo de interfaces y direccionamiento

La divergencia evolutiva entre la familia de sistemas operativos POSIX y la arquitectura Windows NT ha forjado paradigmas marcadamente contrastantes en la interacción usuario-núcleo y la estructuración de recursos. Aunque ambas plataformas proveen entornos multitarea y multiusuario, sus premisas discrepan en el tratamiento de flujos de datos, la instanciación de procesos y la semántica de direccionamiento en almacenamiento secundario (Silberschatz et al., 2006, §2.3, p. 51).

10.1. Comparativa entre entornos de línea de comandos

El modelo del shell en GNU/Linux concibe los componentes como archivos y privilegia flujos continuos de bytes en texto plano sin formato (*unformatted text streams*) comunicados mediante tuberías unidireccionales (`|`) (Stallings, 2018, §1.4, p. 38). En contraparte, `cmd.exe` en Windows NT opera como un invocador de funciones del subsistema Win32,

interactuando mediante búferes de consola orientados a celdas administrados por ConPTY / `conhost.exe`. En el ciclo de vida de procesos, Linux adopta la bifurcación asíncrona con copia en escritura (`fork()`) y carga con `execve()` (Carretero Pérez et al., 2001, §3.2, p. 122), permitiendo aislar y reconfigurar descriptores antes de ejecutar el binario; por el contrario, Windows NT implementa la creación ejecutiva monolítica y atómica mediante `NtCreateUserProcess()` (encapsulada en `CreateProcessW()`), asignando espacio virtual, mapeando el ejecutable PE y enlazando manejadores en una única llamada de alta complejidad (Deitel et al., 2004, §21.4, p. 1044).

10.2. Diferencias operativas en la gestión de rutas

En el sistema de archivos, GNU/Linux articula un árbol unificado en el VFS desde una raíz canónica (`/`), integrando particiones y sistemas sintéticos (`/proc`, `/sys`) mediante puntos de montaje transparentes (Silberschatz et al., 2006, §13.2, p. 455). Por el contrario, Windows NT particiona el espacio en letras de unidad (`C:\`, `D:\`) mapeadas en `\DosDevices`, manteniendo en el bloque PEB un CWD independiente por volumen. Asimismo, el VFS en POSIX es estrictamente sensible a mayúsculas (*case-sensitive*), usa la barra ordinaria (`/`) y mitiga condiciones de carrera mediante descriptores relativos (`openat()`), mientras que el subsistema Win32 es insensible a mayúsculas (*case-insensitive*), emplea la barra invertida (`\`) y valida derechos de acceso evaluando listas DACL de forma centralizada al despachar `NtCreateFile()`.

La Tabla 9 resume las diferencias estructurales y operativas entre los entornos POSIX / GNU/Linux y la arquitectura Windows NT.

Tabla 9: Matriz comparativa multidimensional entre entornos POSIX y Windows NT.

Criterio de diseño	Paradigma POSIX / GNU/Linux	Paradigma Windows NT (CMD / Win32)
Filosofía de comunicación	Flujos de bytes continuos en texto plano sin estructura (`stdout`, tuberías anónimas).	Mensajes de eventos, búferes de pantalla de consola (`conhost.exe`) y API Win32.
Modelo de procesos	Bifurcación desacoplada `fork()` con copia en escritura (*COW*) y carga con `execve()`.	Creación monolítica atómica mediante `CreateProcessW()` / `NtCreateUserProcess()`.
Jerarquía de directorios	Árbol único unificado bajo la raíz canónica `/` mediante puntos de montaje del VFS.	Espacio de nombres particionado en letras de unidad (`C:\`, `D:\`) mapeadas en `\DosDevices`.
Sintaxis y sensibilidad	Sensible a mayúsculas (*case-sensitive*); delimitador canónico de niveles `/`.	Insensible a mayúsculas (*case-insensitive*); delimitador `\` y caracteres reservados.
Gestión del CWD	Puntero único a estructura `struct path` en el descriptor de proceso (`task_struct`).	Múltiples CWD por unidad lógica persistidos en el bloque PEB del proceso.
Resolución y seguridad	Resolución iterativa por inodos; mitigación TOCTOU vía descriptores relativos (`openat`).	Resolución MFT y comprobación centralizada de listas DACL/SACL mediante `NtCreateFile`.

11. Conclusiones personales

El análisis sistemático de la interacción entre el usuario y el sistema operativo desarrollado en este trabajo pone de manifiesto que las interfaces contemporáneas, lejos de ser meras capas superficiales de presentación visual o interpretación sintáctica, constituyen extensiones directas de la arquitectura interna del núcleo. La gestión de atajos de teclado ilustra con notable claridad la intrincada colaboración asíncrona entre el silicio y el espacio de usuario: desde la generación de códigos de escaneo binarios en el controlador físico y el despacho de interrupciones de hardware hacia las rutinas ISR del procesador, hasta la normalización de códigos virtuales en mitades inferiores del kernel y su posterior enrutamiento por parte del compositor gráfico o el subsistema de ventanas. Este flujo evidencia cómo las decisiones de diseño a bajo nivel determinan de manera tajante la seguridad y la latencia del sistema; la transición desde la arquitectura histórica de X11 hacia compositores Wayland y portales de escritorio XDG fundamentados en aislamiento de memoria y paso de mensajes mediante D-Bus erradica vectores de intercepción pasiva (*keylogging*) que durante décadas comprometieron la integridad del escritorio en entornos UNIX, equilibrando la inmediatez operativa con el confinamiento de privilegios.

En el ámbito de la edición interactiva de código y configuración de infraestructura, el estudio profundo del editor Vi y su evolución hacia Vim revela la vigencia y superioridad ergonómica del paradigma modal frente al modelo de inserción continua dominante en procesadores de texto y entornos integrados modernos. Concebir el editor como una máquina de estados finitos determinista sustentada en una gramática formal rigurosa transforma la edición en un lenguaje algebraico donde los operadores léxicos interactúan armónicamente con objetos de texto y cuantificadores numéricos. Al confinar los desplazamientos y comandos a la fila central del teclado alfanumérico, el modelo modal suprime drásticamente la sobrecarga biomecánica provocada por la conmutación constante hacia el ratón o el uso forzado de teclas modificadoras periféricas, reduciendo la fricción cognitiva en sesiones prolongadas de programación y administración remota. Asimismo, el desacoplamiento entre el búfer dinámico en memoria RAM y el archivo de intercambio temporal en disco ejemplifica una implementación magistral de tolerancia a fallos y concurrencia mediante cerrojos a nivel de proceso, garantizando la consistencia transaccional del código incluso ante anomalías intempestivas del entorno de ejecución.

La exploración comparativa de las interfaces de línea de comandos en GNU/Linux y Microsoft Windows permite dimensionar dos filosofías arquitectónicas divergentes pero altamente refinadas para el control de recursos. En el ecosistema POSIX, el principio fundamental de estructurar la comunicación mediante flujos unidireccionales de bytes en texto plano sin formato, concatenados mediante tuberías y primitivas estándar de entrada y salida, propicia la modularidad y la composición ortogonal de herramientas especializadas. Este enfoque, complementado por la bifurcación asíncrona de procesos mediante `fork()` bajo el esquema de copia en escritura y la posterior carga con `execve()`, otorga una flexibilidad granular insuperable para la automatización de tareas complejas en el centro de datos. En contraste, el intérprete `cmd.exe` y el subsistema Win32 abordan la interacción a través de búferes de pantalla de consola administrados por procesos anfitriones y la invocación atómica de funciones ejecutivas monolíticas en el núcleo Windows NT. Si bien este modelo demanda estructuras de datos más rígidas y una sobrecarga superior en la inicialización de procesos mediante `NtCreateUserProcess()`, ofrece una integración estrecha y determinista con el modelo de objetos, los descriptores de seguridad y las listas de control de acceso propias de la infraestructura corporativa de Microsoft.

Finalmente, la formalización técnica del direccionamiento jerárquico subraya el papel preponderante del sistema de archivos como organizador del estado persistente de la máquina. La confrontación entre el árbol unificado bajo la raíz canónica del VFS en GNU/Linux y el particionamiento en letras de unidad independientes en Windows NT refleja visiones ontológicas distintas sobre la abstracción del hardware. Mientras el VFS integra discos y recursos sintéticos bajo descriptores seguros (`openat()`) para evitar condiciones de carrera temporales, Windows mantiene contextos de trabajo independientes por volumen en el bloque PEB y valida los accesos mediante tablas MFT y el Gestor de Objetos. Dominar la distinción entre rutas absolutas invariantes y relativas contextuales constituye una competencia crítica de ingeniería para diseñar software portable, robusto y protegido contra ataques de secuestro de rutas.

Referencias

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson Prentice Hall.

Flatpak and Freedesktop.org Contributors. (2024a). *Xdg desktop portal: \NoCaseChange{org.freedesktop.portal.GlobalShortcuts}*. Consultado el 15 de septiembre de 2026, desde https://flatpak.github.io/xdg-desktop-portal/docs/doc-org.freedesktop.portal.GlobalShortcuts.html

Flatpak and Freedesktop.org Contributors. (2024b). *Xdg desktop portal: \NoCaseChange{org.freedesktop.portal.Screenshot}*. Consultado el 15 de septiembre de 2026, desde https://flatpak.github.io/xdg-desktop-portal/docs/doc-org.freedesktop.portal.Screenshot.html

IEEE and The Open Group. (2018). *Standard for Information Technology—Portable Operating System Interface (POSIX(R)) Base Specifications, Issue 7: vi utility* (inf. téc. N.º IEEE Std 1003.1-2017). IEEE. Consultado el 15 de septiembre de 2026, desde https://pubs.opengroup.org/onlinepubs/9699919799/utilities/vi.html

Microsoft Corporation. (2023a). *About keyboard input*. Consultado el 15 de septiembre de 2026, desde https://learn.microsoft.com/en-us/windows/win32/inputdev/about-keyboard-input

Microsoft Corporation. (2023b). *Registerhotkey function (winuser.h)*. Consultado el 15 de septiembre de 2026, desde https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-registerhotkey

Microsoft Corporation. (2024). *Keyboard shortcuts in windows*. Consultado el 15 de septiembre de 2026, desde https://support.microsoft.com/en-us/windows/keyboard-shortcuts-in-windows-dcc61a7a-8320-bba5-1256-f0ba0bef256b

Moolenaar, B., & The Vim Development Group. (2024). *Vim online documentation: Introduction and user manual*. Consultado el 15 de septiembre de 2026, desde https://www.vim.org/docs.php

Oualline, S. (2001). *Vi improved—vim*. New Riders Publishing.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

The GNOME Project. (2024). *Useful keyboard shortcuts*. Consultado el 15 de septiembre de 2026, desde https://help.gnome.org/users/gnome-help/stable/shell-keyboard-shortcuts.html